# SISTEMAS TOLERANTES A FALLOS

*Grado en Ingeniería Informática*

Introducción a los Sistemas Operativos en Tiempo Real (RTOS)

Prof. Jose Antonio Huerga

*Lo que ya conocéis…*

# Programación mediante ‘Super lazos’ (Super loops / Single Threaded)

# Foreground-Background Systems

**Low Priority** - - -

# Arquitectura Super-Loop y Foreground-Background

- Arquitectura *Super-loop*:

- No hace uso de interrupciones, por lo que el monitoreo de eventos se realiza mediante Polling
- Se trata de un único hilo o Thread de ejecución, por lo que no existen variables o datos compartidos con otros hilos.
- Se aplica a sistemas simples.
- Pueden usarse funciones de tipo 'non-reentrant'. Una función non-reentrant es aquella función que no puede ser compartida por más de un hilo o tarea, a menos que exista algún método de exclusión (mecanismos thread-safe como mutex o secciones críticas), por miedo a que los datos queden corruptos durante operaciones RW/WR. Puesto que nos encontramos en un sistema que dispone de un solo hilo y sin interrupciones, podemos hacer uso de este tipo de funciones con seguridad. Este tipo de funciones no deben usarse en sistemas concurrentes o pseudo concurrentes en los que existen varios hilos ejecutándose al mismo tiempo.
- Las latencias o tiempos de respuesta de estos sistemas son poco predecibles. En estos casos, la latencia hace referencia al tiempo que transcurre desde que se produce un evento hasta que se inicia la función asociada a ese evento. Por otro lado, el tiempo de respuesta tiene también en cuenta el delay que toma la función asociada en finalizar su ejecución.
- Al ser una arquitectura muy simple, implica que cualquier cambio de código puede alterar en gran medida el comportamiento general del sistema y sus tiempos de respuesta.
- Atribuido a lo anterior, el sistema es muy dependiente de la estructura general del código y de la secuencia de eventos.

# Arquitectura Super-Loop y Foreground-Background

- Arquitectura *Foreground-Background*:

- Se trata de la arquitectura *Super-Loop* junto con el uso de interrupciones.
- Las interrupciones forman la parte Foreground, donde se puede crear un sistema basado en prioridades (varios hilos).
- En el background disponemos de un único hilo de baja prioridad, con funciones no críticas en tiempo
- Los hilos pueden comunicarse entre si mediante variables globales. Hay que tener cuidado con las secciones críticas e interrupciones anidadas.
- En caso de realizar todo el procesamiento en el background, implica que todas las funciones presentan la misma prioridad. En este caso, se simula un super-loop
- En caso de procesar gran parte de las funciones en el Foreground, implica que las interrupciones se alargan y por tanto se penaliza la capacidad de respuesta general del sistema, sobre todo para interrupciones de baja prioridad.

# Arquitectura Super-Loop y Foreground-Background

- Función de tipo Reentrant y Non-Reentrant

En sistemas en los que se dispone de un único procesador, una función se denomina reentrante cuando puede ser invocada con seguridad en un sistema concurrente.

Por ejemplo, si nos encontramos en medio de la ejecución de esta función y otro proceso interrumpe la ejecución actual e invoca de nuevo esta función (*re-entered*) de manera segura (sin afectar a los datos de la ejecución anterior), podemos decir que la función es de tipo *reentrant.*

Las restricciones básicas para considerar una función como *reentrant* son:

- La función no debe contener datos estáticos (*static data*) durante llamadas sucesivas.
- No debe retornar un puntero que apunte a un dato estático, si no que todos los datos deben ser proporcionados por el procedimiento que invoca a la función.
- Debe usar variables locales. En caso de usar variables globales, debe tener un mecanismo de protección haciendo una copia de la misma.
- No debe llamar a funciones no-reentrantes.

# Arquitectura Super-Loop y Foreground-Background

- Función de tipo Non-Reentrant: Static Data

Caso 1

```c
#include<stdio.h>
int fun()
{
  static int count = 0;
  count++;
  return count;
}

int main()
{
  printf("%d ", fun());
  printf("%d ", fun());
  return 0;
}
```

Caso 2

```c
#include<stdio.h>
int fun()
{
  int count = 0;
  count++;
  return count;
}

int main()
{
  printf("%d ", fun());
  printf("%d ", fun());
  return 0;
}
```

# Arquitectura Super-Loop y Foreground-Background

- Función “Non-Reentrant” y código “Non-Thread-Safe”

```
int tmp;

void swap(int* x, int* y)
{
    tmp = *x;
    *x = *y;
    /* Hardware interrupt might invoke isr() here. */
    *y = tmp;
}

void isr()
{
    int x = 1, y = 2;
    swap(&x, &y);
}
```

# Arquitectura Super-Loop y Foreground-Background

- Función “Non-Reentrant” y código “Thread-Safe”

```
_Thread_local int tmp;

void swap(int* x, int* y)
{
    tmp = *x;
    *x = *y;
    /* Hardware interrupt might invoke isr() here. */
    *y = tmp;
}

void isr()
{
    int x = 1, y = 2;
    swap(&x, &y);
}
```

# Arquitectura Super-Loop y Foreground-Background

- Función “Reentrant” y código “Thread-Safe”

```
void swap(int* x, int* y)
{
    int tmp;
    tmp = *x;
    *x = *y;
    *y = tmp;    /* Hardware interrupt might invoke isr() here. */
}

void isr()
{
    int x = 1, y = 2;
    swap(&x, &y);
}
```

# APLICACIONES EN TIEMPO REAL (RTA)

- *Conceptos erróneos sobre la computación en tiempo real:*

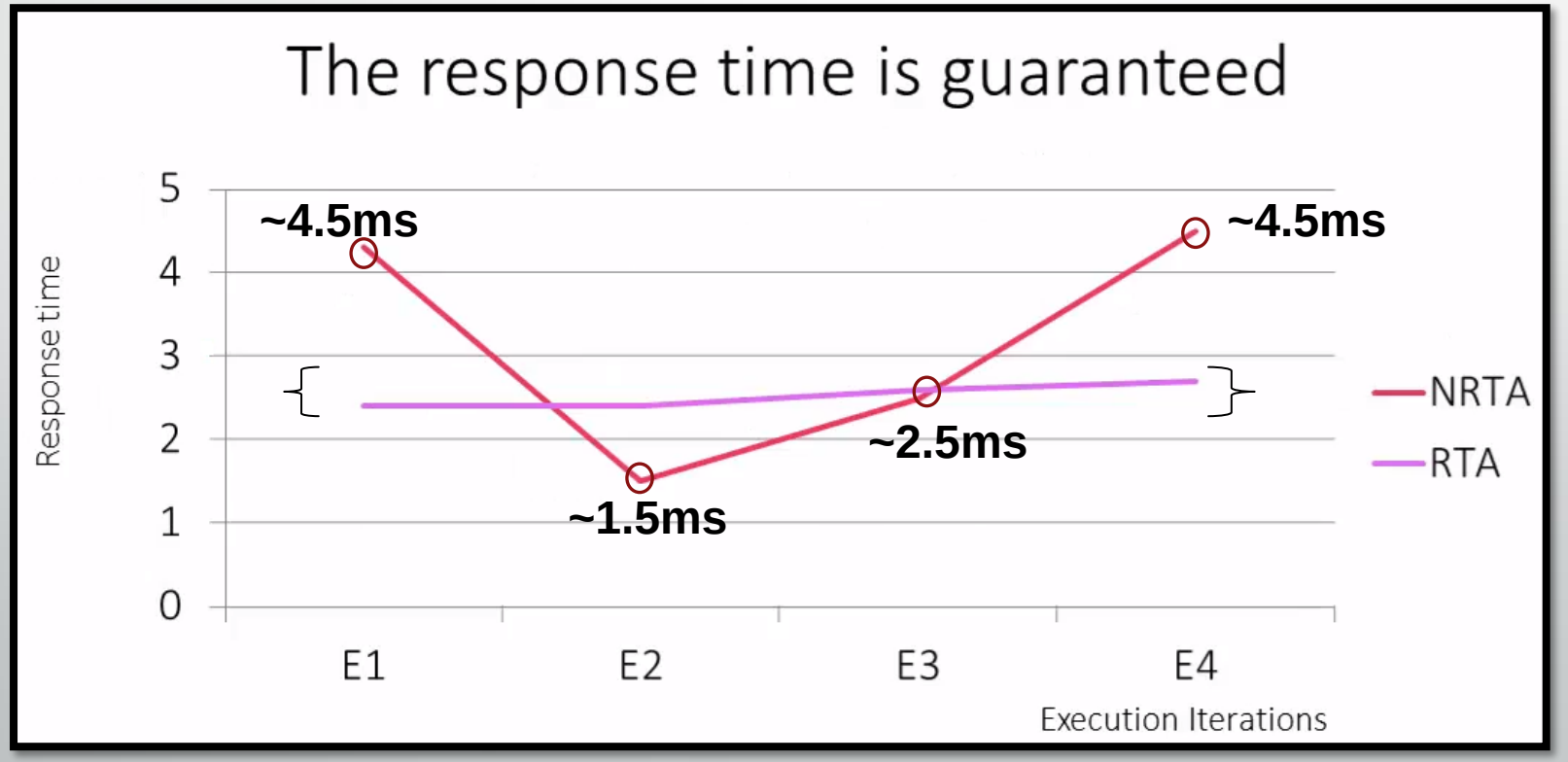
  - La computación en tiempo real es equivalente a procesar líneas de código a alta velocidad.
  - En las RTA se obtiene mayor rendimiento (*Mayor cantidad de procesos por unidad de tiempo*).
  - Disponer de más procesadores, memoria RAM, e interfaces de bus veloces permite transformar nuestro sistema a una RTA

- *La realidad sobre la computación en tiempo real:*

  - La computación en tiempo real es equivalente a computación PREDECIBLE.
  - En las RTA los tiempos de criticidad son mas importantes que el rendimiento.
  - Un sistema en tiempo real trabaja con GARANTIAS, no promesas. Disponer de altas velocidades de computación no garantiza que una aplicación funcione en tiempo real.

# ¿Qué es Tiempo Real (Real-Time)?

*"Un sistema en Tiempo Real es aquel en el que la exactitud del computo realizado no sólo depende de la exactitud lógica del cómputo, si no también del instante de tiempo en el que se produjo. Si las restricciones de tiempo no se cumplen, se dice que el sistema ha fallado."*



# APLICACIONES EN TIEMPO REAL (RTA)

- *Características principales:*

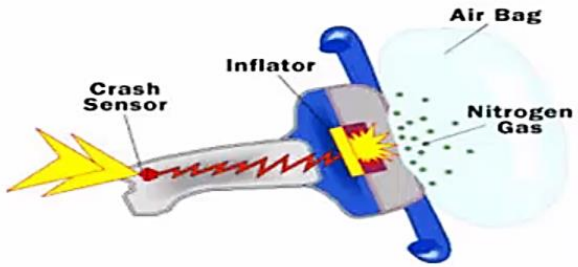
- Las RTAs no son aplicaciones que se ejecuten rápidamente.
- Son aplicaciones determinadas en el tiempo. Sus tiempos de respuesta son prácticamente constantes.
- Pueden existir pequeñas desviaciones ocasionales en el tiempo de respuesta, en términos de milisegundos o segundos. Estas aplicaciones caen en la categoría de "*Soft RTAs*". (*E.g. Sistema de adquisición de datos*).
- Por otro lado, los sistemas "*Hard RTAs*" deben cumplir todas sus funciones en un tiempo muy estricto. El hecho de fallar en este requerimiento puede producir un fallo absoluto/catastrófico en el sistema. (*E.g. sistema de control de vuelo*).
  - En esta categoría tenemos los sistemas "Real RTAs", en el que los tiempos de respuesta son aún más restrictivos. (*E.g. Sistema de guiado de misiles*).

# APLICACIONES EN TIEMPO REAL (RTA)

- *Hard RTA vs Soft RTA:*

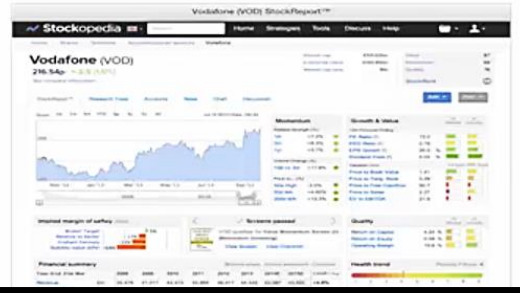
GUIADO DE MISILES

SISTEMA DE FRENOS ANTIBLOQUEO




AIRBAG

TELEFONÍA Y COM.




WEB DE ADQUISICIÓN DE DATOS

# APLICACIONES EN TIEMPO REAL (RTA)

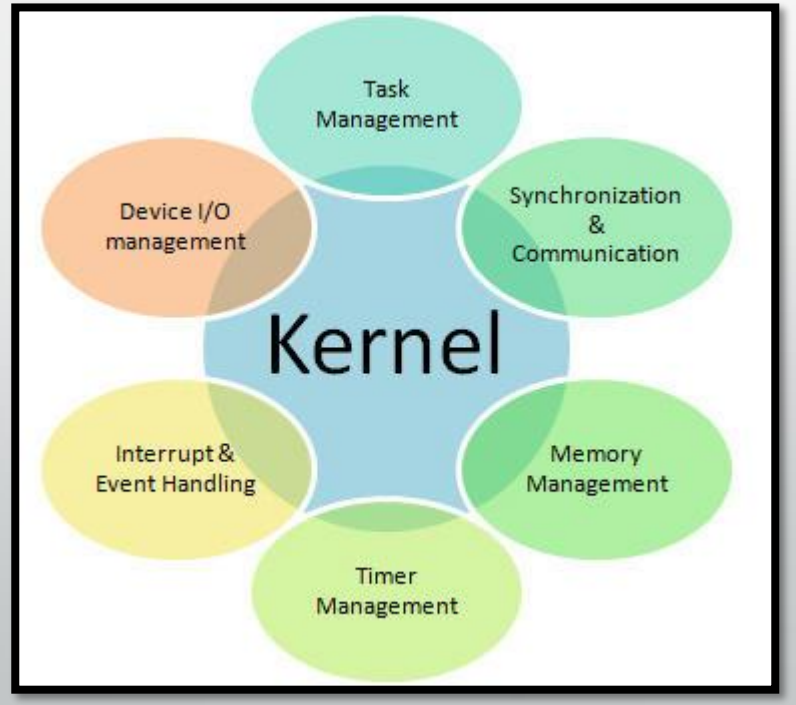
- *Aplicaciones en varios sectores*

| | |
|---|---|
| Aeroespacial | Detección de fallos (Hard RTA)<br>Controlador de vuelo (Hard RTA)<br>Display (Soft RTA) |
| Finanzas | ATM (Hard RTA)<br>Web mercado de valores (Soft RTA) |
| Medicina | Escáner Rayos X (Hard RTA) |
| Industrial | Automatización sistema de fabricación PCB (Hard RTA)<br>Estación meteorológica (Soft RTA) |
| Comunicaciones | Audio o Streaming de video (Soft RTA) |

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

*¿ Cómo programamos una aplicación en tiempo real (RTA)?*

**Necesitamos un Sistema Operativo en Tiempo Real**



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *Características principales:*

- Un RTOS está especialmente diseñado para procesar aplicaciones que requieran tiempos precisos con un alto grado de fiabilidad y tiempos predecibles y determinísticos.
- Para ser considerado como "tiempo-real", el sistema operativo **debe** saber el tiempo máximo de ejecución de cada operación crítica. En esta categoría tenemos:
  - Procesamiento de las interrupciones y excepciones internas del sistema (System Exceptions)
  - Secciones críticas (Critical Sections): proteger zonas críticas ante un posible context switch o interrupción.
  - Mecanismo de planificación del Kernel (Scheduling): Preemptive (expropiativo), cooperativo, modalidad Round-Robin.
  - Objetos de kernel: Queue, Semáforos binarios, Mutexes, etc.

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Planificador de tareas (Task Scheduling)*

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Planificador de tareas (Task Scheduling)*

  - Los GPOS están programados para planificar las tareas de modo que se consiga una alta cantidad de procesos en una unidad de tiempo. Este parámetro se conoce como *Throughput*.
  - En muchas ocasiones, la ejecución de los procesos de alta prioridad se puede ver retrasada con el fin de procesar un mayor número de tareas de menor prioridad. Esto se consigue procesando varias tareas de baja prioridad en lugar de la tarea de alta prioridad.

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Planificador de tareas (Task Scheduling)*
- Por otro lado, los *RTOS* ejecutan sus procesos en función de su prioridad. Si una tarea (*Task*, *Thread*) esta lista para ser ejecutada (***Ready to Run state***), tomará el control de la *CPU* de cualquier otra tarea de menor prioridad que esté ejecutándose (***Switch-Context***)
- Todas las tareas de baja prioridad se mantienen en pausa (***Suspended***) si existe una tarea de alta prioridad lista para ejecutarse. En este caso, una tarea de alta prioridad solo puede verse anulada por otras tareas de prioridad superior o interrupciones (*ISRs*).
- En cuanto a la tasa de procesamiento de tareas por unidad de tiempo (*Throughput*), los *RTOS* muestran una menor tasa que los *GPOS*, debido a la "*restricción*" de ejecutar siempre la tarea de mayor prioridad. Sin embargo, este hecho no implica que los RTOS tengan un *Throughput* escaso.
- Un RTOS de calidad deberá ser capaz de ofrecer un *Throughput* decente, pero puede sacrificarlo a cambio de ser determinista y predecible en el tiempo.

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Planificador de tareas (Task Scheduling)*

- En los RTOS, ser predecibles y determinísticos (***Predictability*** and ***Deterministic***) es más importante que lograr un buen ***Throughput***.
- Para el caso de los GPOS, lograr altas tasas de procesamiento resulta más importante para la conveniencia del usuario.

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Latencia (Latency)*

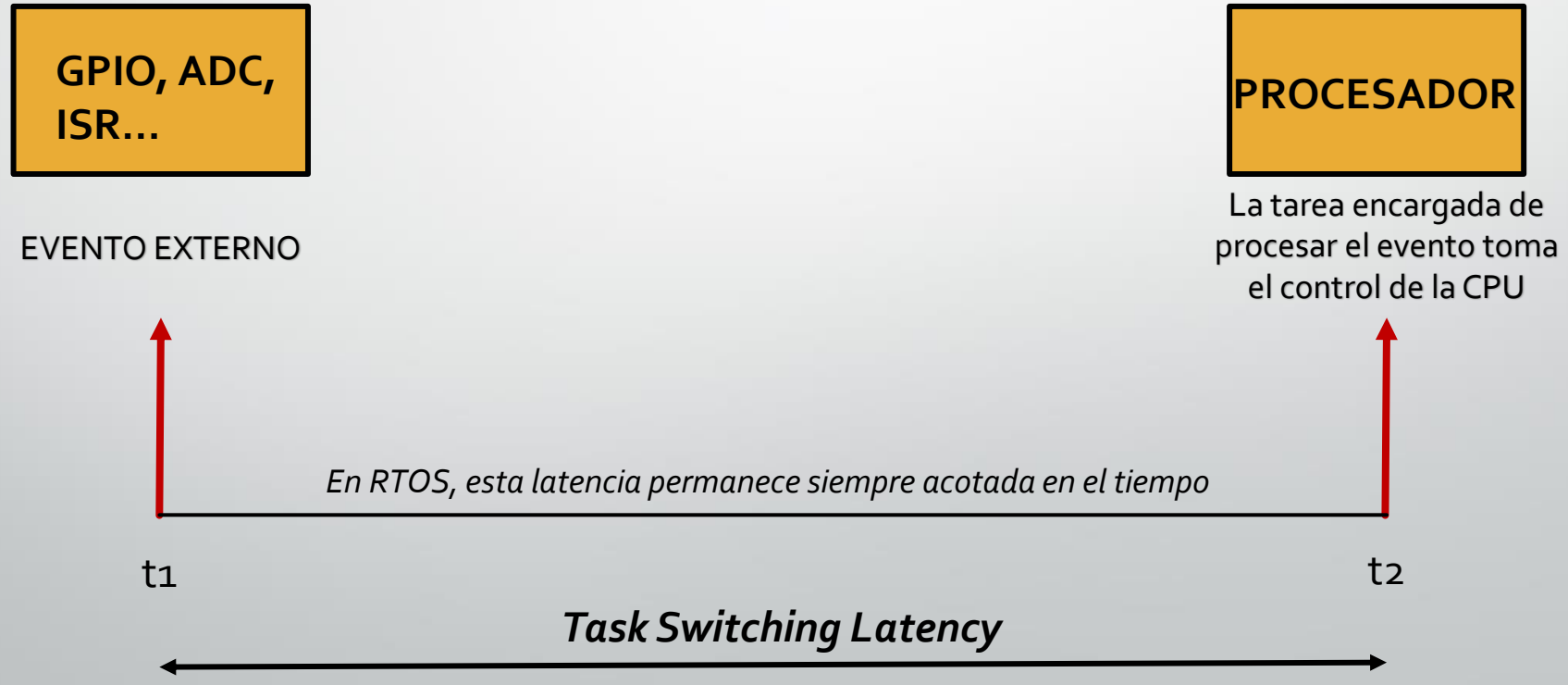
- En el ámbito de la computación, la latencia hace referencia al *tiempo que transcurre entre un estímulo y la ejecución de la función asociada.*
- En *RTOS*, cuando hablamos de conmutación entre tareas (***Task Switching***), la latencia hace referencia a los ciclos de reloj transcurridos entre un evento y el tiempo que toma la tarea en tomar el control de la *CPU* para poder procesar dicho evento. (No se tiene en cuenta el tiempo de procesamiento después de haber tomado el control de la *CPU*).

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Latencia (Latency)*

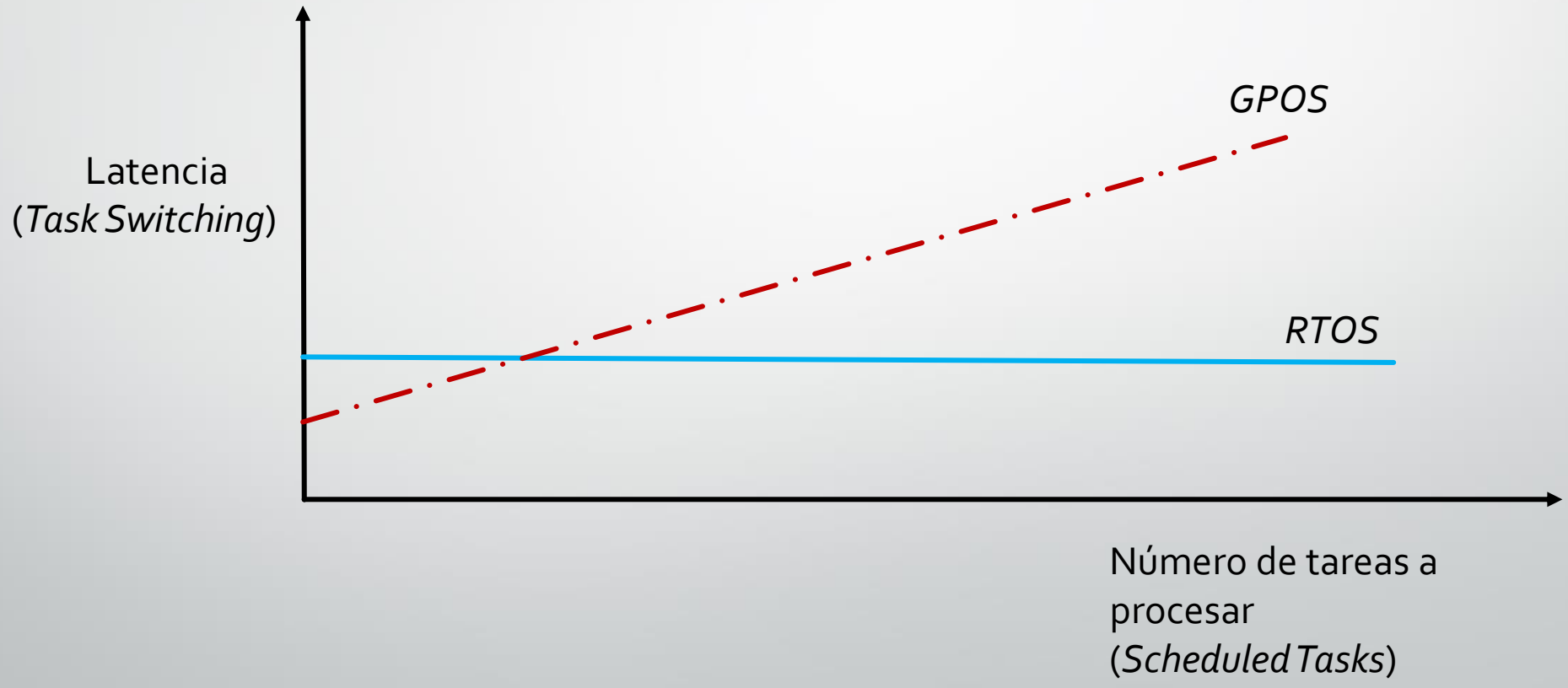
En los *GPOS*, el tiempo de latencia puede variar significativamente.

En los *RTOS*, el tiempo de latencia permanece prácticamente constante.



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

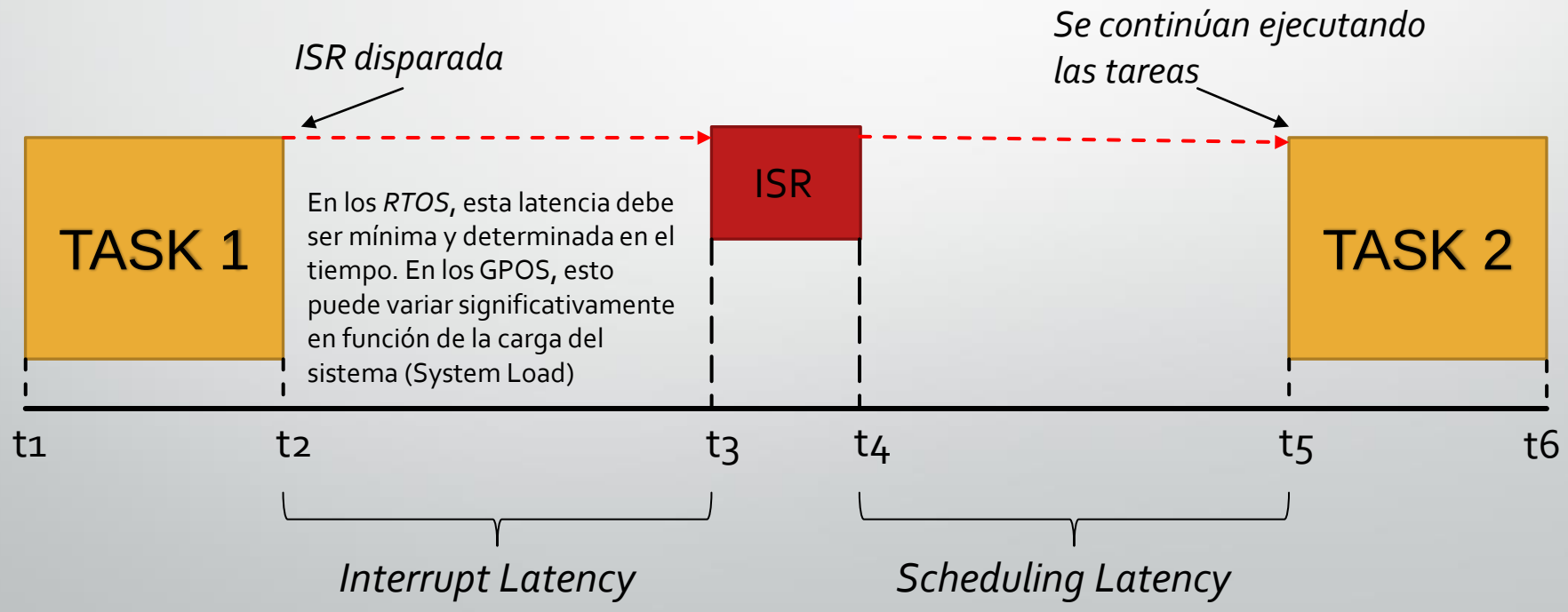
- *RTOS vs GPOS: Latencia (Latency)*



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Latencia (Latency)*

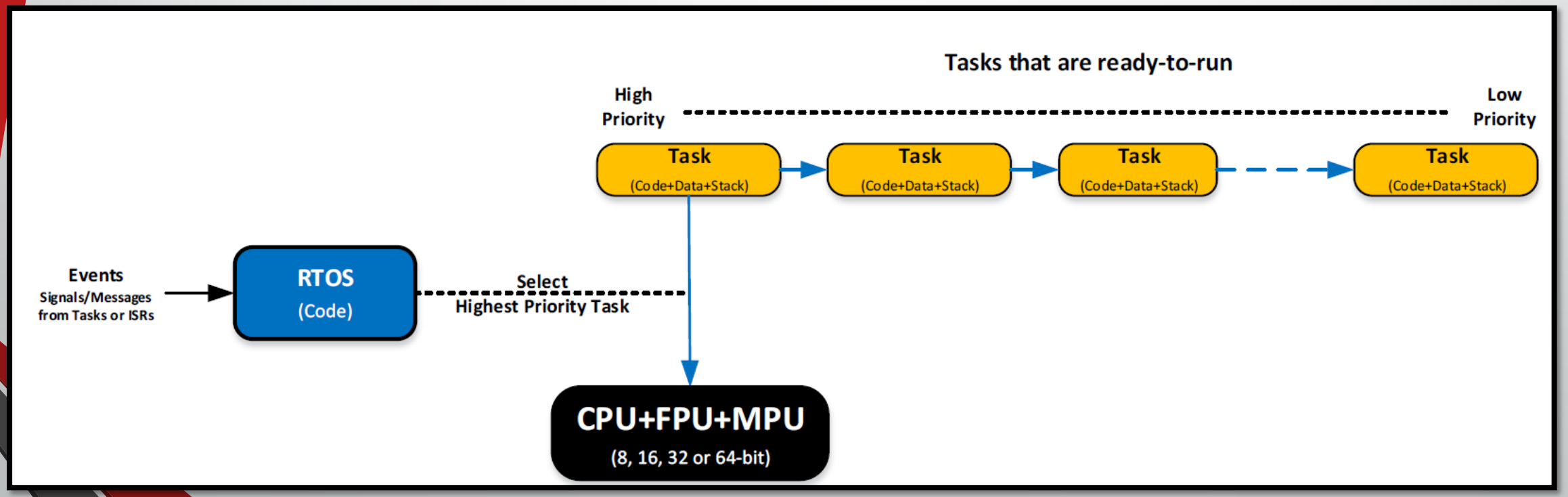
En el anterior apartado se ha explicado el concepto de latencia entre tareas (*Task Switching*). Otro parámetro importante a tener en cuenta es la latencia a la hora de procesar una interrupción (***Interrupt latency***)



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

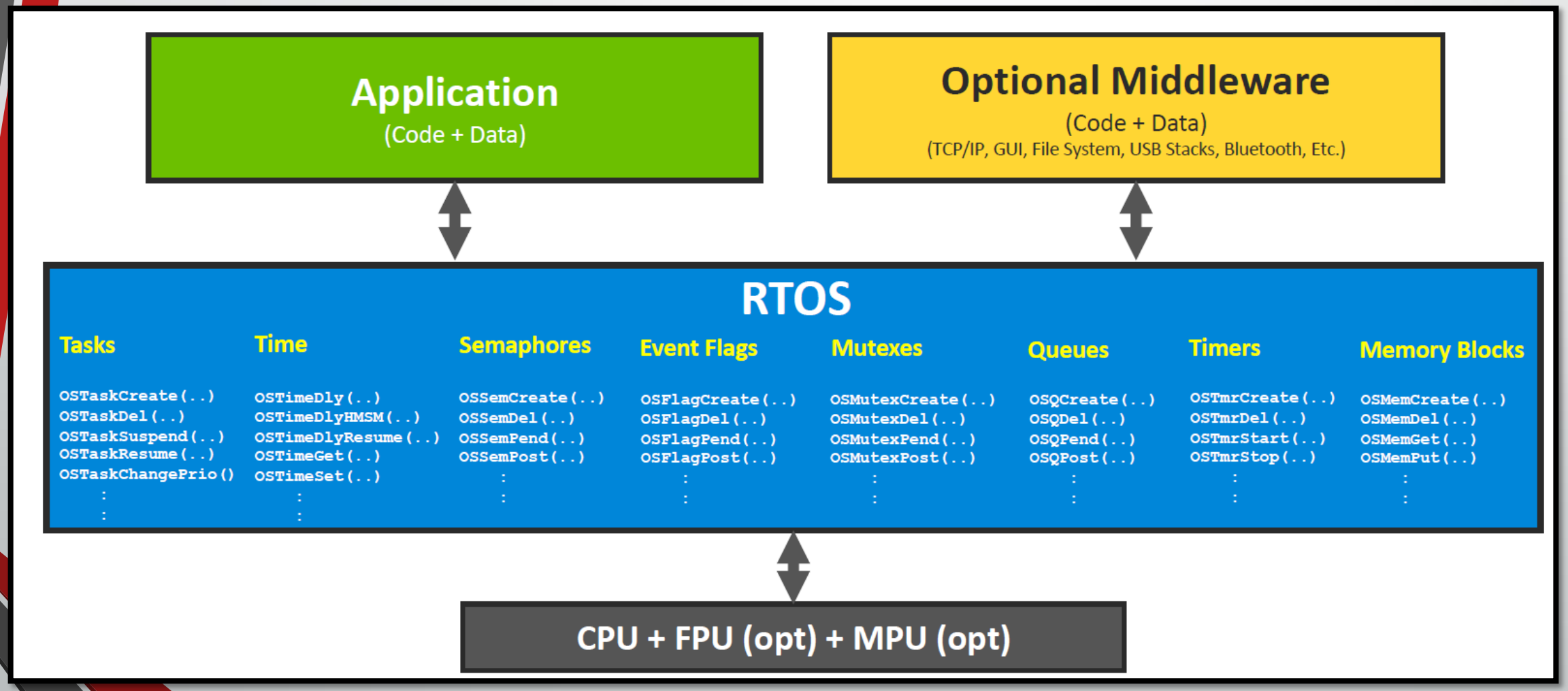
Un **RTOS** es básicamente software que se encarga de gestionar el tiempo y los recursos de la CPU:

- La aplicación se divide en múltiples tareas
- El objetivo principal del **RTOS** es ejecutar la tarea más importante que este lista para ser ejecutada (*Ready to Run state*).
- En sistemas con única CPU (microcontrolador generalmente), sólo puede estar procesandose una tarea para un instante de tiempo determinado.



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

Servicios que ofrece el **RTOS** a nuestra aplicación



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

Un **RTOS** presenta los siguientes beneficios:

- Permite dividir la aplicación en tareas, las cuales pueden ser configuradas por prioridad de manera individual.
- Añadir tareas de baja prioridad (*Low Priority*) no afecta a la capacidad de respuesta de las tareas de alta prioridad (*High priority*).
- Las tareas pueden esperar por un evento o eventos (*wait event*) para ser procesadas. Esto evita hacer uso de polling.
- El código middleware puede añadirse al sistema con facilidad.
- Permite crear un entorno de trabajo (*framework*) que facilita el desarrollo de software entre varios programadores o un equipo de trabajo.
- Mayor control de los plazos de desarrollo de una aplicación (*deadlines*).
- La mayoría de sistemas operativos en el mercado han sido revisados y testeados. Muchos de ellos con certificación para un entorno o industria. En este sentido, es difícil encontrar bugs en el software de un RTOS.
- Permiten portabilidad a diferentes arquitecturas o CPUs. El fabricante debe dar indicaciones de los puertos permitidos (microcontroladores, fpgas…)
- Resulta muy sencillo gestionar los recursos energía. Sleep modes, Suspended states….

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

Un **RTOS** presenta los siguientes desventajas:

- Al ser código extra que debe ser añadido al sistema, se requiere mayor memoria FLASH. Típicamente entre 6 a 20 *Kbytes*.
- Precisamos mayor memoria ***RAM***, ya que cada *Task* requiere de su propia ***stack*** de *RAM* individual. Este tamaño será configurable y depende de la aplicación y los recursos que deba gestionar la propia tarea.
- Cada tarea debe ser asignada a un bloque de control de tareas ***TCB*** propio (***T****ask* ***C****ontrol* ***B****lock*). Este bloque de control es usado por el ***Kernel*** para almacenar la información particular de cada tarea, dentro del espacio de memoria del usuario (*RAM*).
- Aproximadamente unos 256 bytes de memoria *RAM* para variables propias del RTOS.
- La asignación de prioridades a cada ***Task*** es siempre necesaria. El problema es que esta asignación a veces no es trivial y puede resultar complicado decidir que prioridad se debe dar.
- Los servicios de un RTOS consumen ciclos de CPU y añaden carga de trabajo al sistema, típicamente del 2% al 10%.
- Para cada RTOS seleccionado, existe una curva de aprendizaje diferente.

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

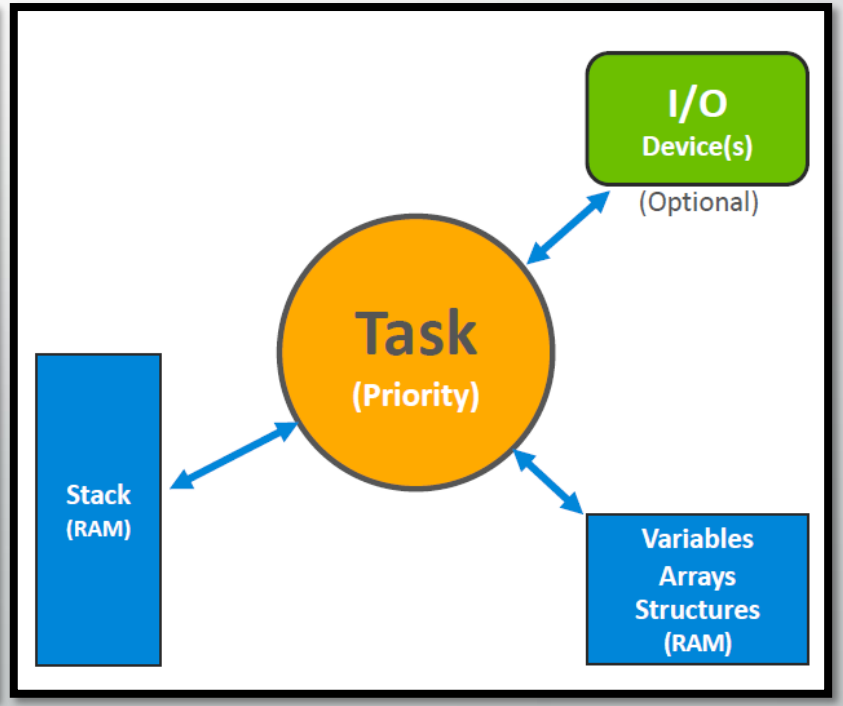
En cuanto a las ***Tasks***:

- Se debe añadir la configuración de prioridad, individualmente.
- Requiere su propia Stack (RAM).
- Gestiona sus propias variables, arrays y estructuras.
- Cada Task se configuran típicamente en bucle infinito.
- En el código de cada Task se puede añadir software de gestión de dispositivos I/O.
- Contiene una parte del código de tu aplicación.

```
CPU_STK  MyTaskStk[MY_TASK_STK_SIZE];   // Task Stack


void MyTask (void *p_arg)               // Task Code
{
  Local Variables;

  Task initialization;
  while (1) {                           // Infinite Loop (Typ.)
    Wait for Event;
    Perform task operation;             // Do something useful
  }
}
```

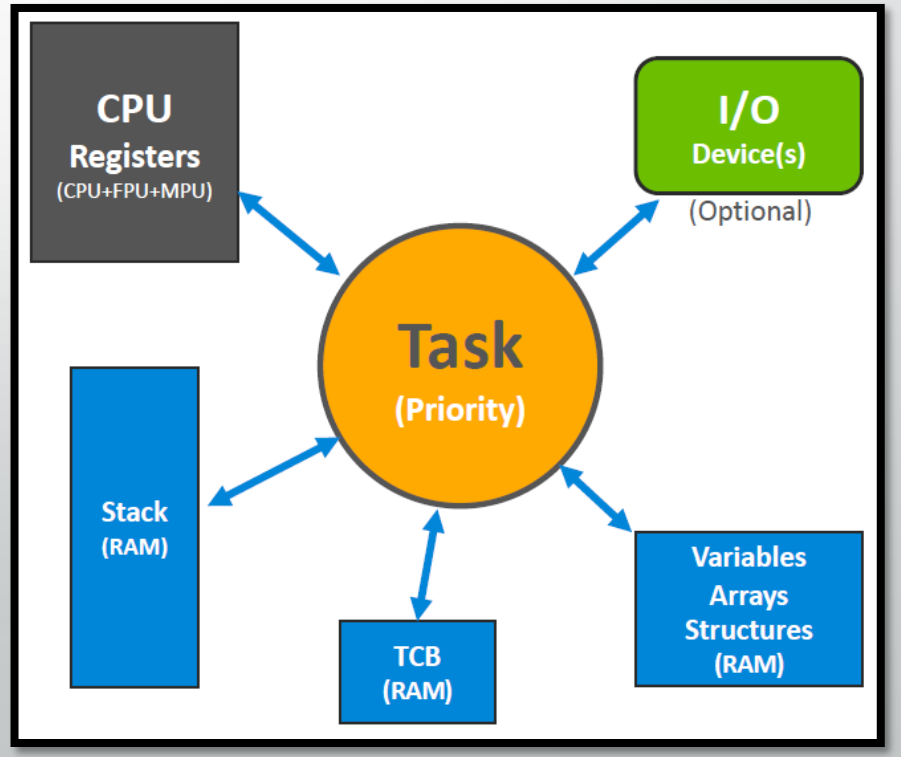


# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

En cuanto a la creación de ***Tasks***:

- El RTOS dispone de una API o función de creación de Tasks. Esto hace que el sistema tenga constancia de que queremos crear una tarea, con una configuración determinada.
- El RTOS asigna a cada tarea su propio acceso a un conjunto de registros de CPU, FPU (*Floating Point Unit*), MPU (*Mathematical Unit*), etc…
- El RTOS asigna a cada Task su propio TCB (*Task Control Block*).

```
void OSTaskCreate (MyTask,            // Address of code
                   &MyTaskStk[0],     // Base of stack
                    MY_TASK_STK_SIZE,// Size of stack
                    MY_TASK_PRIO,     // Task priority
                    :
                    :);
```



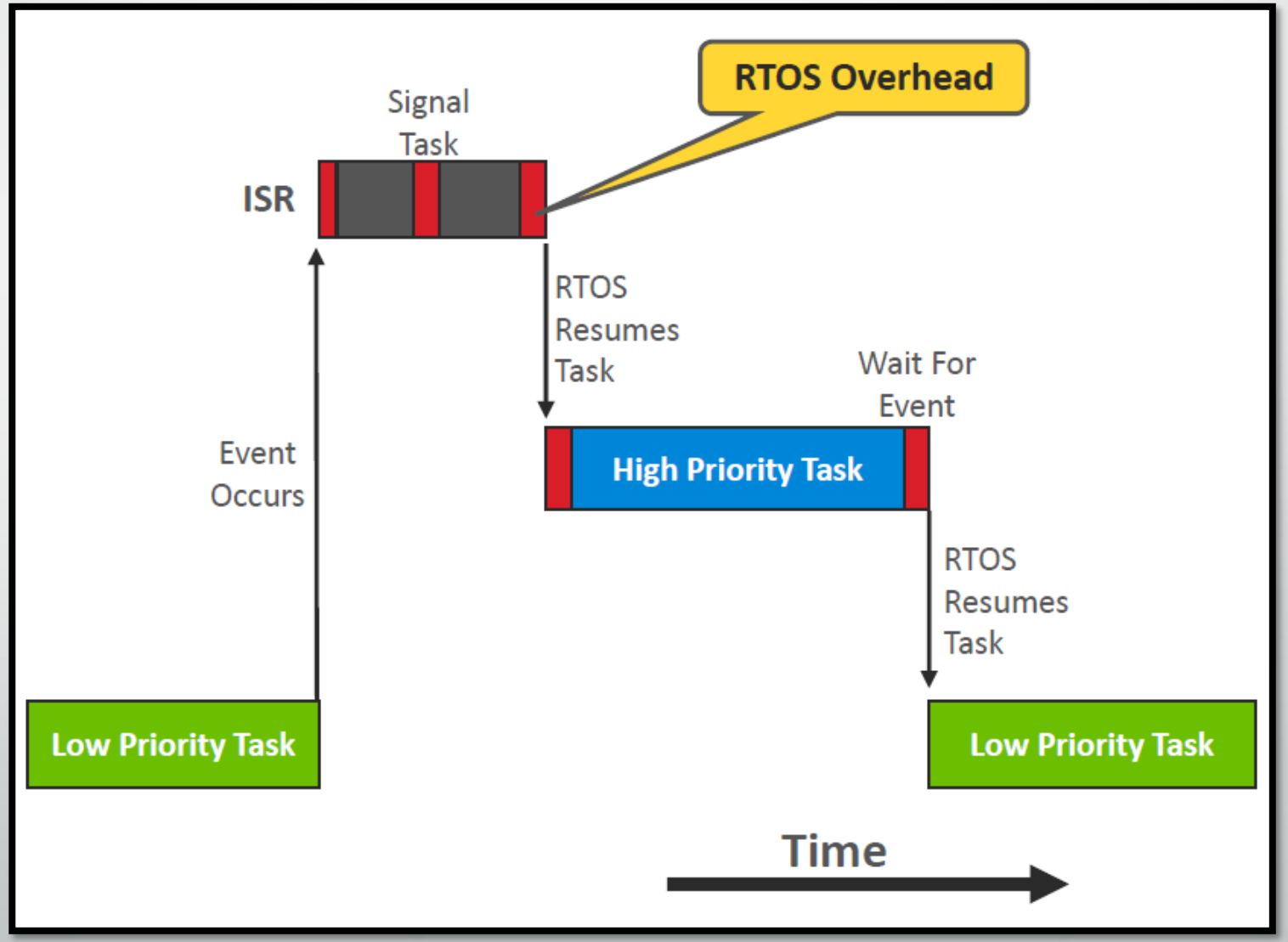
# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

RTOS de tipo ***Preemptive:***

```
void Low_Prio_Task (void)
{
  Task initialization;
  while (1) {
    Setup to wait for event;
    Wait for event to occur;
    Perform task operation;
  }
}
```

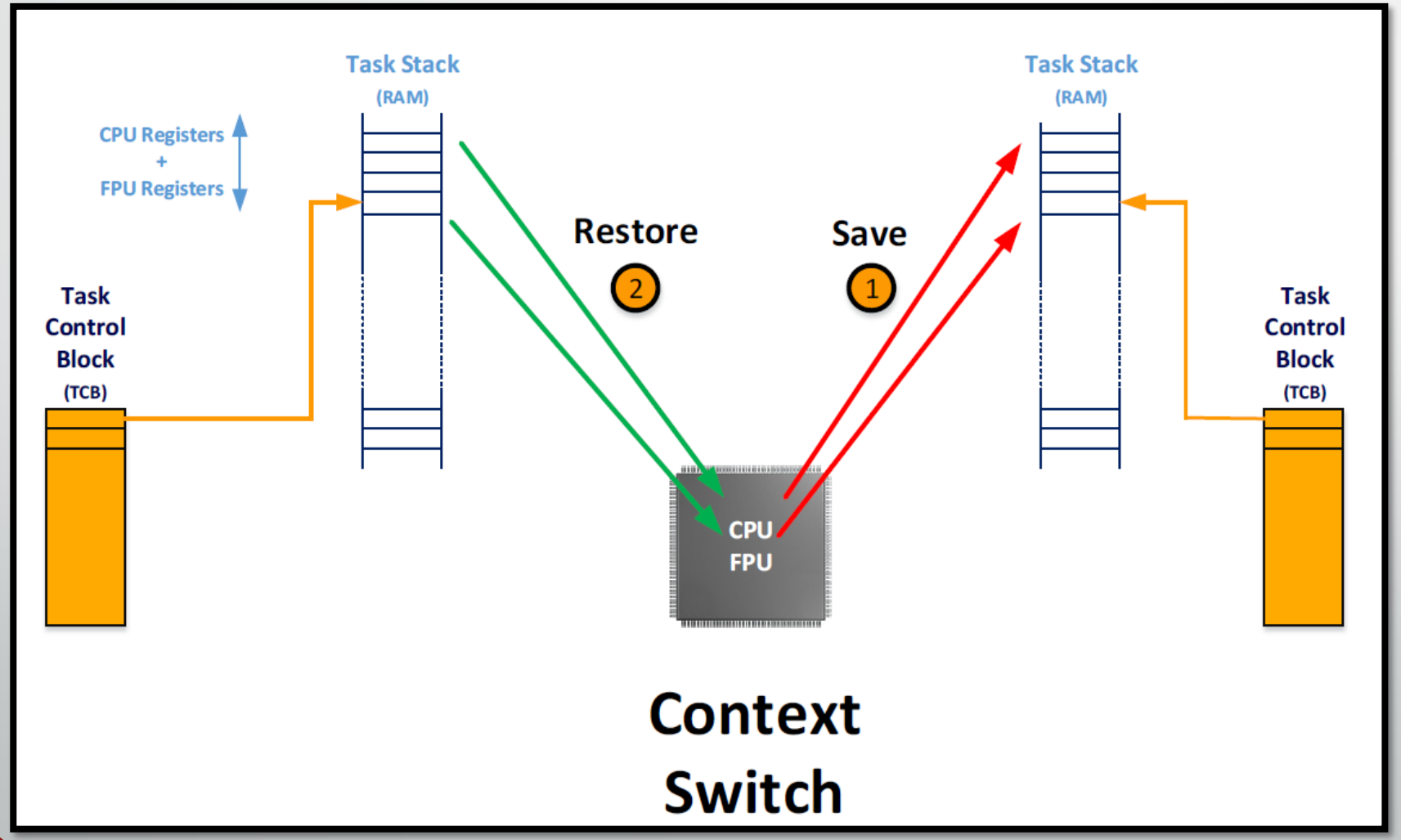
```
void ISR (void)
{
  Entering ISR;
  Perform Work;
  Signal or Send Message to Task;
  Perform Work;  // Optional
  Leaving ISR;
}
```

```
void High_Prio_Task (void)
{
  Task initialization;
  while (1) {
    Setup to wait for event;
    Wait for event to occur;
    Perform task operation;
  }
}
```



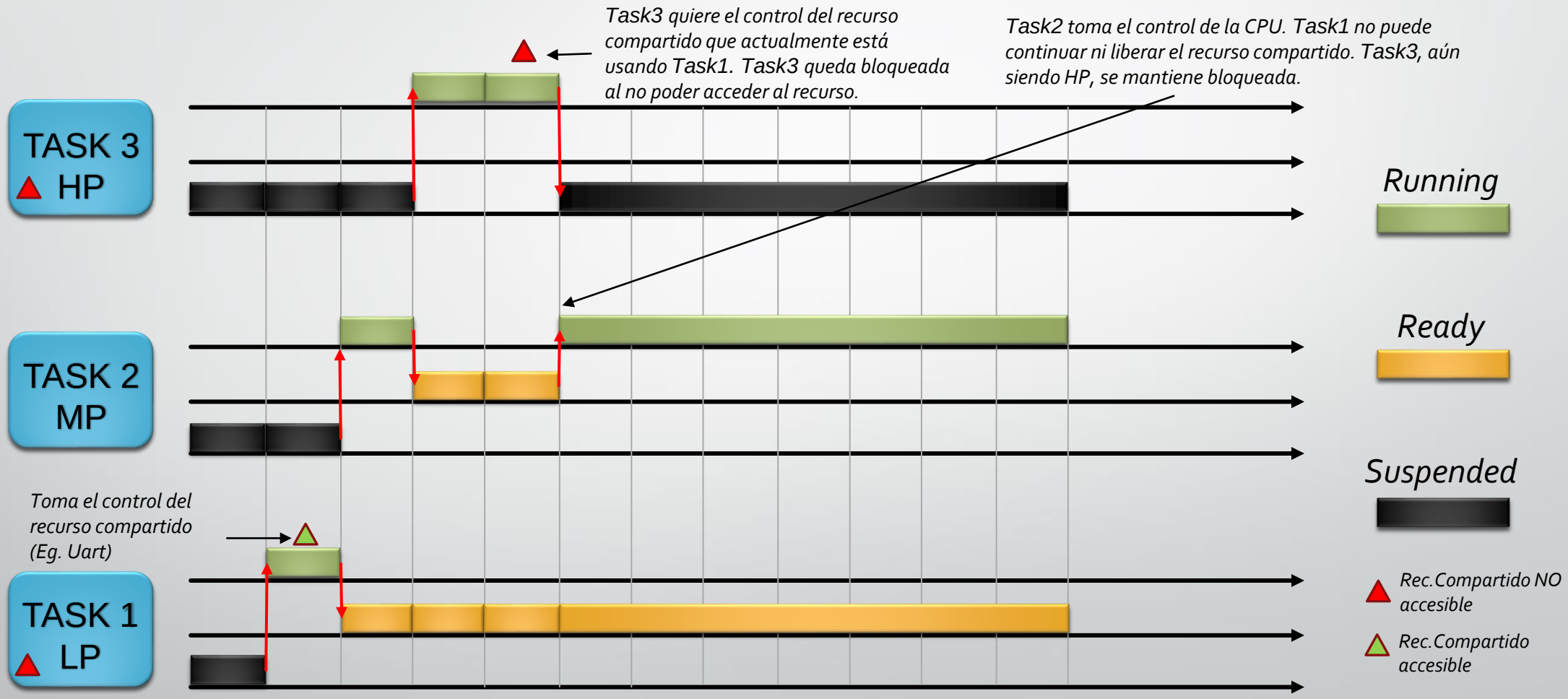
# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

*Context Switch entre Tasks:*



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: Inversión de prioridades (**Priority Inversion**)*



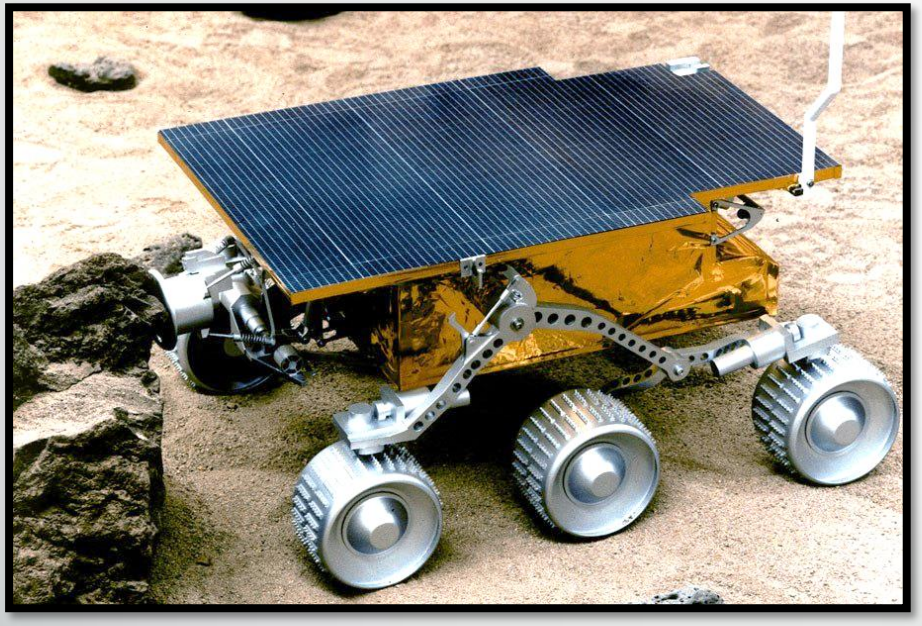
# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

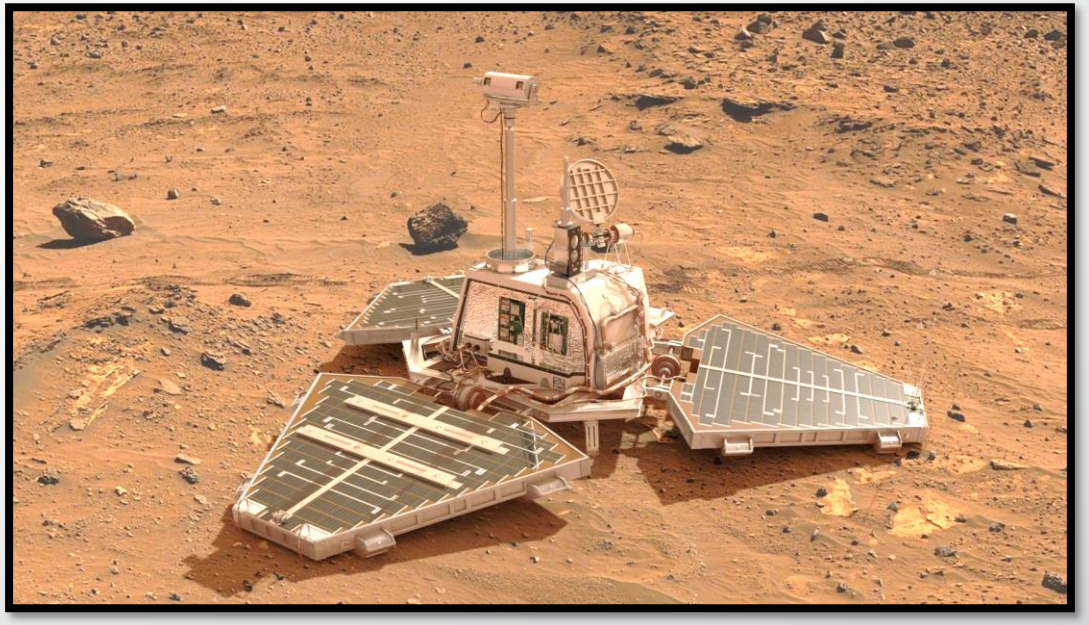
- *RTOS vs GPOS: Inversión de prioridades (**Priority Inversion**)*

- La *inversión de prioridad* se produce cuando una tarea de menor prioridad (*LP*) accede previamente a un recurso compartido (con una tarea de alta prioridad *HP*). En este caso, la tarea de alta prioridad se mantiene bloqueada a la espera de que la tarea de menor prioridad libere el recurso compartido. El efecto producido es el siguiente: ***La tarea de baja prioridad se comporta como una tarea de alta prioridad. Y viceversa, la tarea de alta prioridad se comporta como una de baja prioridad.***
- Una posible solución a este problema es la denominada "Herencia de Prioridades" (***Priority Inheritance***). La tarea de baja prioridad hereda **temporalmente** la prioridad más alta. De este modo se consigue liberar el recurso compartido (***Shared Resource***).

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: (Priority Inversion). Incidente en la misión Mars Pathfinder de la NASA, 1997*

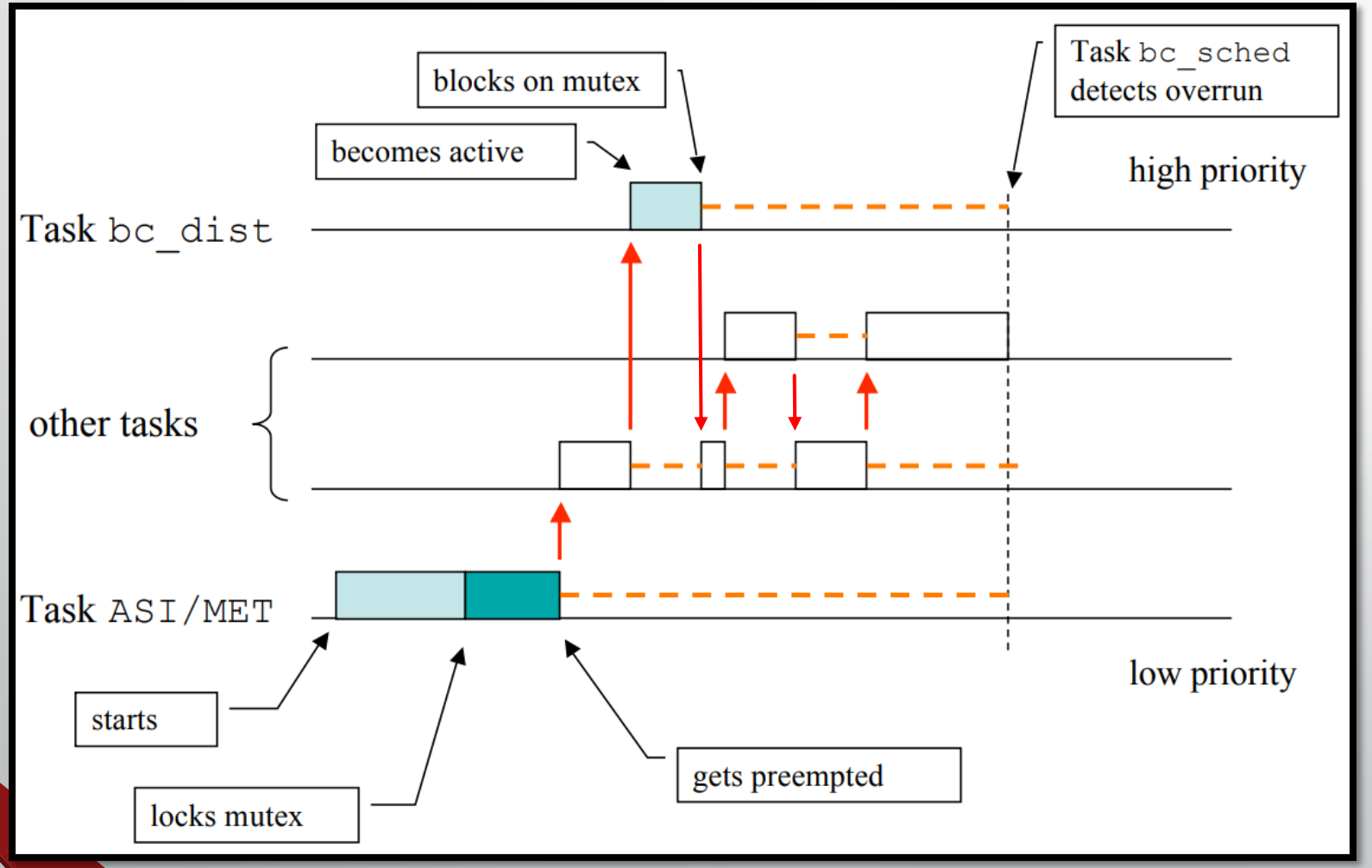




- *Glitches detectados en el software. El pathfinder experimentaba multiples **Resets** a la hora de adquirir datos meteorológicos.*
- *El Reset era provocado por el WatchDog, debido a Timing Overruns de la tarea de alta prioridad.*
- *El problema es conocido como **Priority Inversion**.*

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: (Priority Inversion). Incidente en la misión Mars Pathfinder de la NASA, 1997*




***Task ASI/MET***: Tarea encargada de procesar los datos meteorológicos. Low priority.

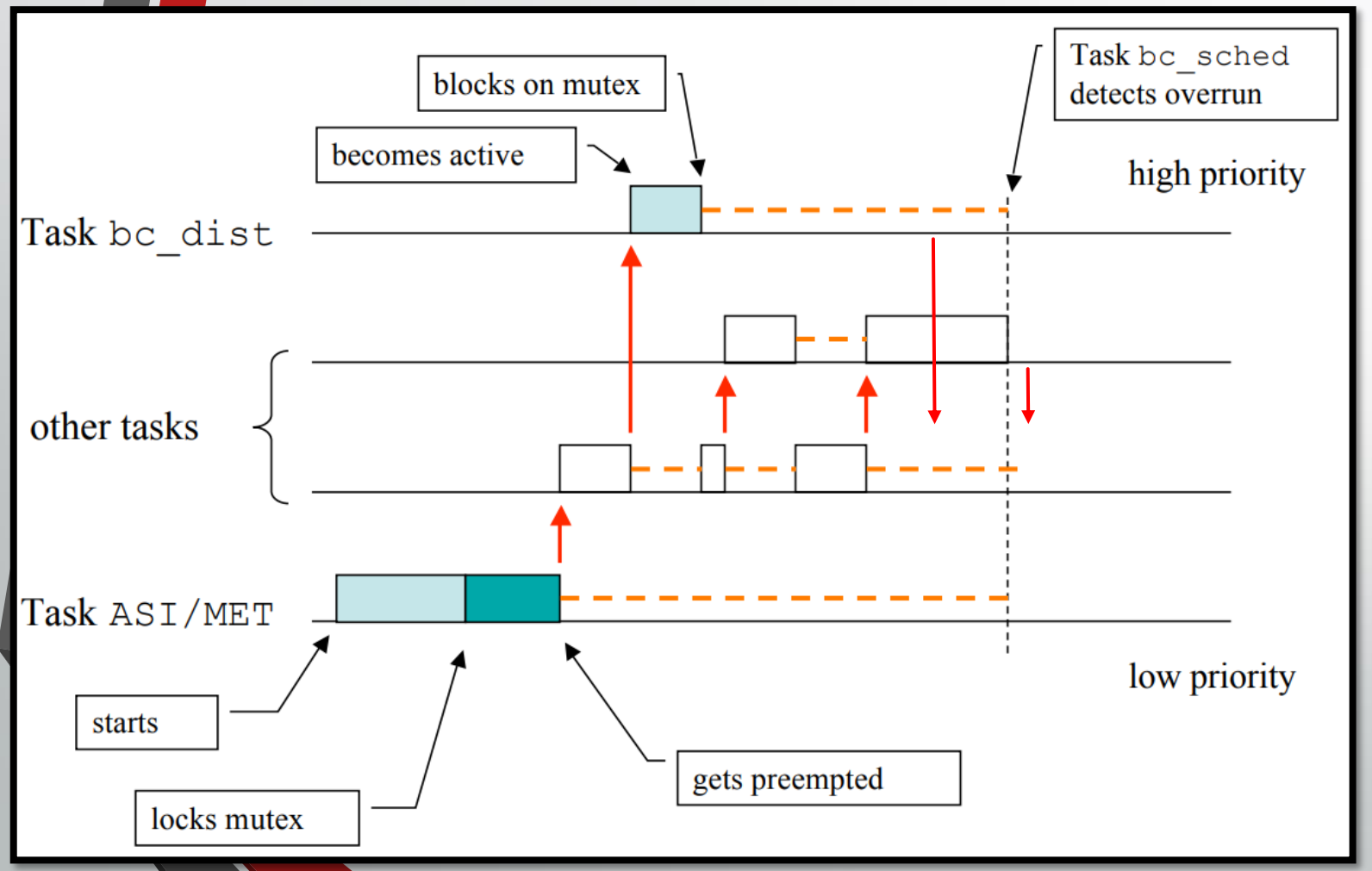
***Task bc_dist*** y ***Task bc_sched***: Tareas críticas y de alta prioridad para controlar el bus de comunicaciones del Lander. Cada tarea era supervisada en cada ciclo para asegurarse de que habían funcionado correctamente y en los tiempos correctos.

*El RTOS usado en la misión fue el VxWorks, de la compañía WindRiver*

VxWorks

# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *RTOS vs GPOS: (Priority Inversion). Incidente en la misión Mars Pathfinder de la NASA, 1997*

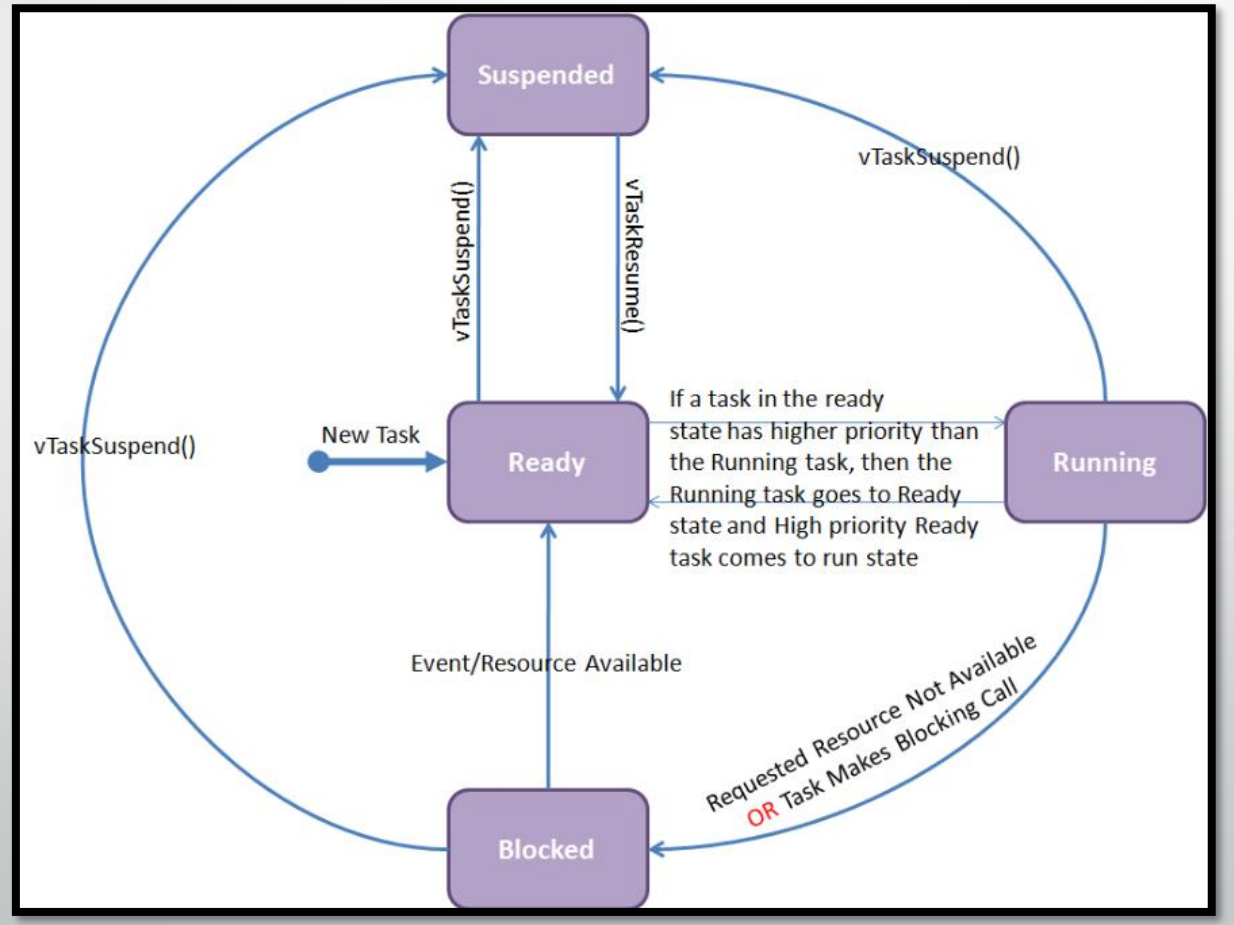



El error, por tanto, se producía por la imposibilidad de ***bc_dist*** de completar su ejecución antes de que **bc_sched** empezase. Puesto que el ciclo de ejecución de cada tarea se comprueba cíclicamente y que trabajamos con un RTOS y una aplicación de tiempo real, el *watchdog* se dispara para llevar al sistema a un estado seguro.

Muchas horas de muestreado de datos de la superficie de Marte se perdían debido a este problema.

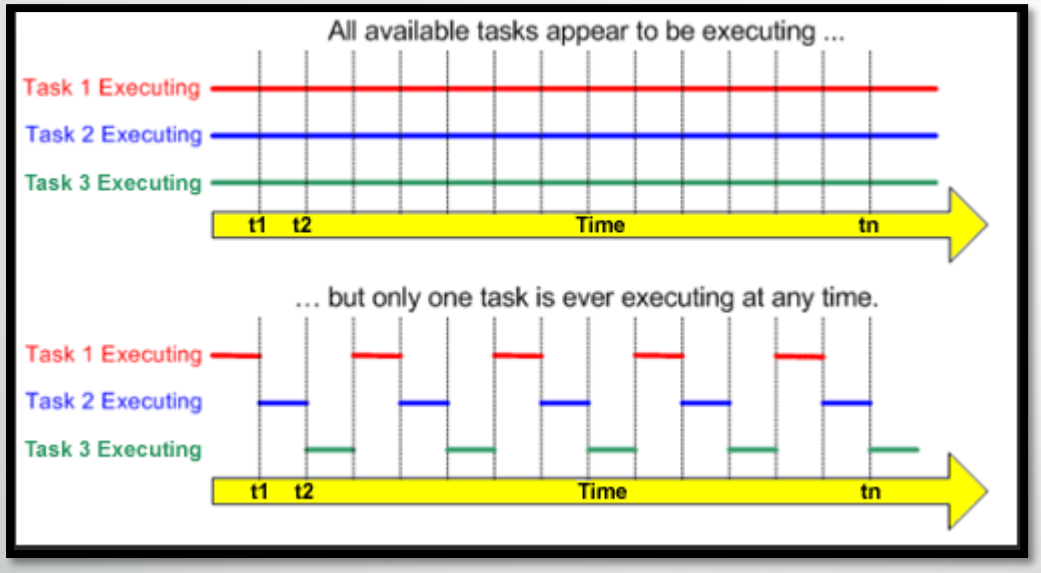
# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

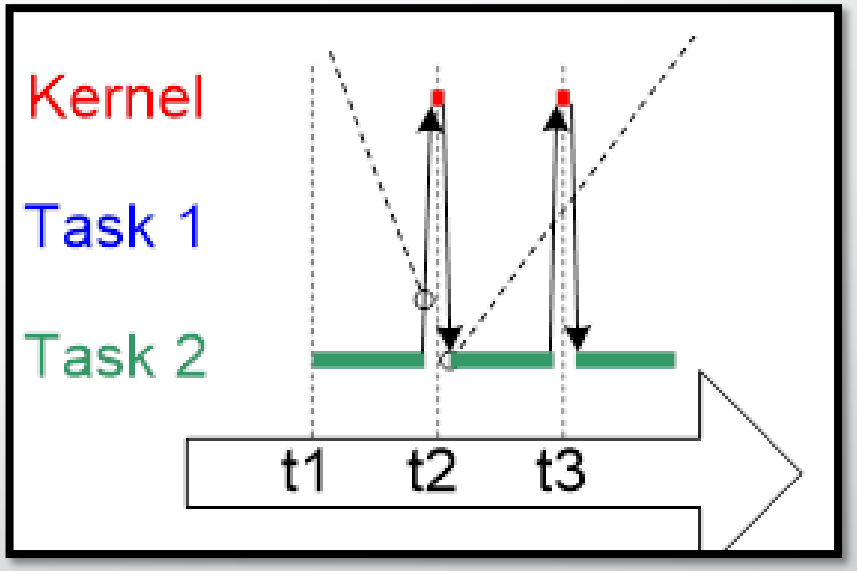
- ***MultiTasking**: Estados posibles de las tareas (Tasks)*



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

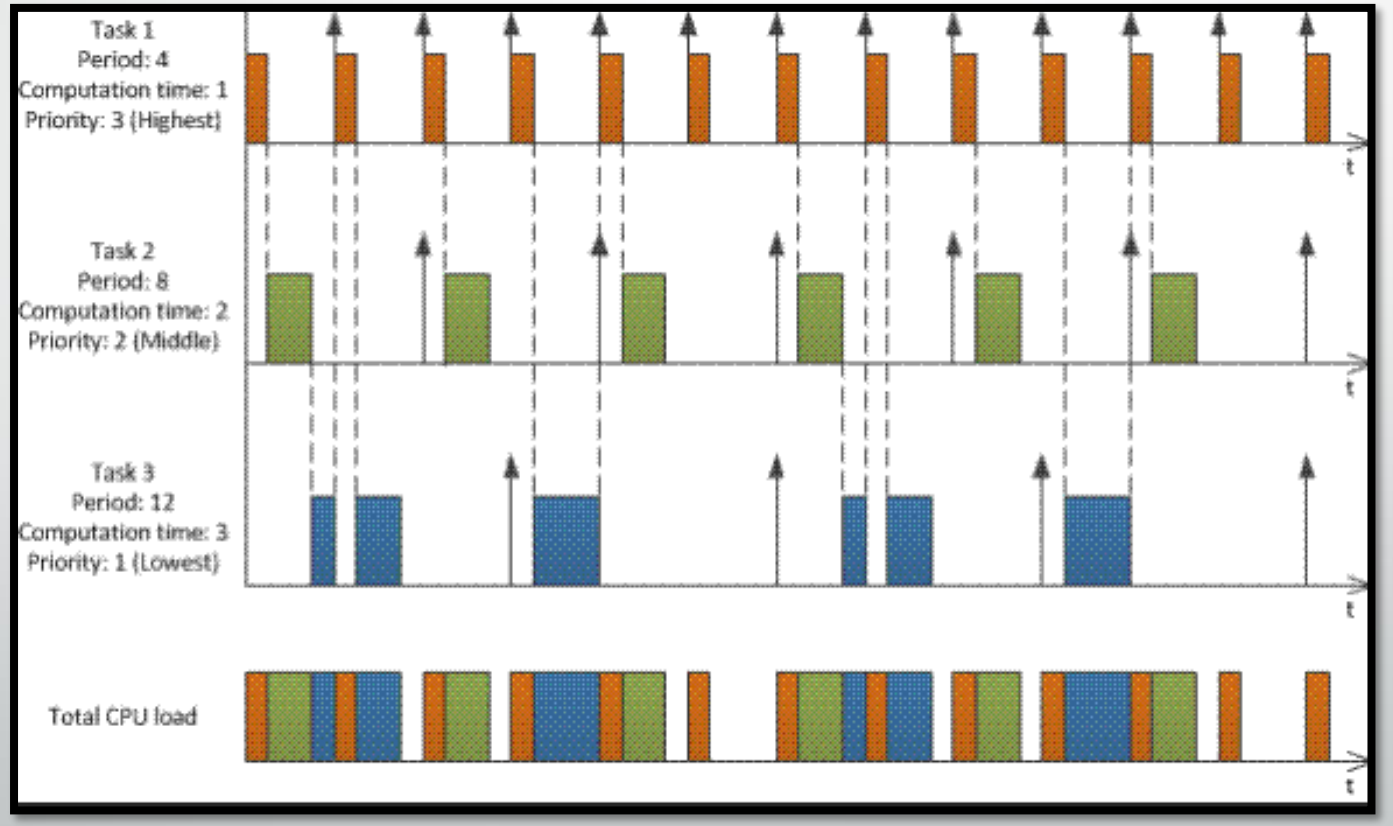
- *Multitasking: Ejemplo básico*

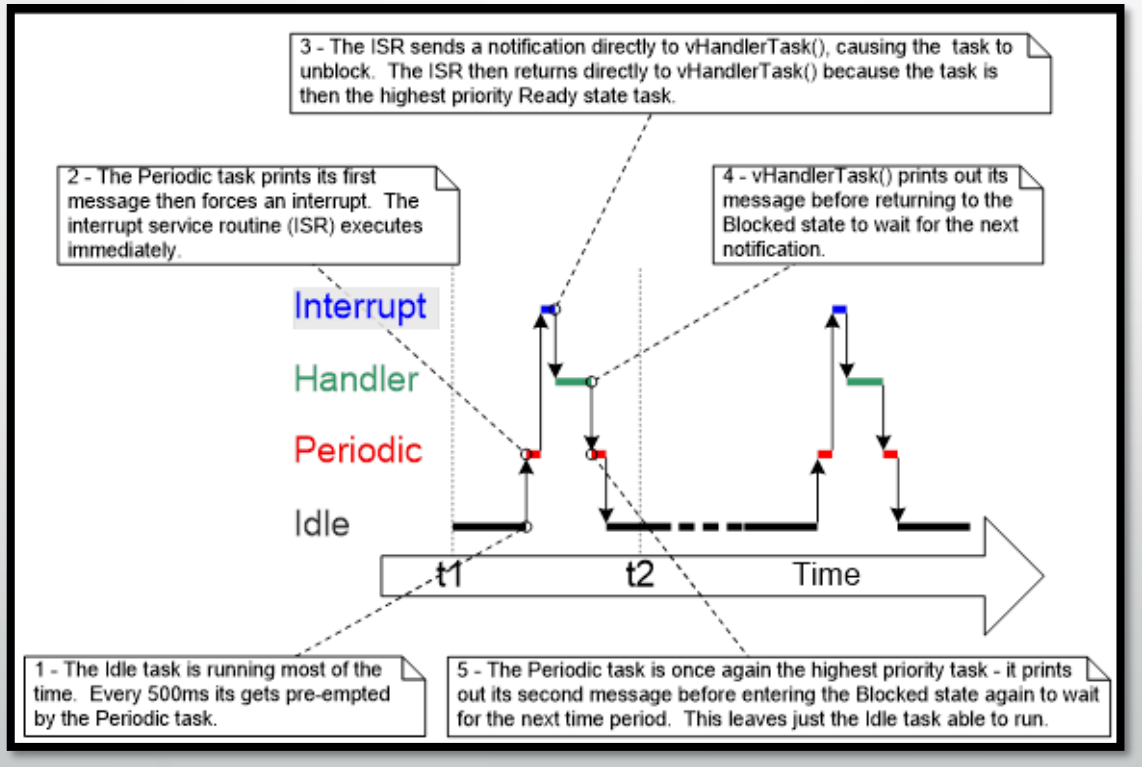


# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *Multitasking: Ejemplo básico*



# SISTEMAS OPERATIVOS EN TIEMPO REAL (RTOS)

- *Multitasking: Ejemplo básico con ISR*